

CAS4160 Homework 5: Offline RL

[Kim Minsung]

[2020142087]

1 Introduction

In this assignment, you will implement one of the fundamental algorithms in offline reinforcement learning: Conservative Q-Learning (CQL). You will be experimenting with different hyperparameters and offline datasets that have been provided to you. In this assignment, we provide a random policy and an expert policy for you to compare the performance according to the quality of the collected data.

Your objectives are as follows:

1. Implement and train an offline reinforcement learning algorithm, CQL
2. Experiment with the key hyperparameters and explore how they affect performance of your RL agent
3. Analyze differences between the quality of different offline datasets, Random and Expert

Writing the report: Provide your responses for questions with %TODO tags in this LaTeX template. Submit all the requested values in tables, and put in all requested plots/images. You should also include all your reasoning and text responses in the PDF.

2 Codebase

In this assignment, you mainly work with the following codes:

- `cas4160/scripts/run_cql.py`: the main training loop for your CQL implementation.
- `cas4160/critics/cql_critic.py`: the CQL learner you will implement.

For each problem, we specify which files you need to complete. Sections that need to be filled have been marked with TODO tags.

Please follow the setup instructions provided in the README.md file to install the dependencies.

3 Tips for experiments

In this homework, our code is computationally light; if you use a RTX 3090 GPU, the GPU utilization will be up to 20%. To utilize the full power of GPUs, we recommend you to run multiple jobs simultaneously. You can do it by (1) executing multiple bash terminals and execute the training script for each bash terminal; (2) using “&” command to run the jobs in background; or (3) using `tmux`. We recommend trying `tmux` since it is one of the powerful utility tools that you can use inside linux servers. One of the advantages of `tmux` is that your work status would be saved inside a `tmux` session, so it will keep running even if you loose the ssh connection.

4 Preliminaries

Environments: In this assignment, you will be working with the Pointmass environment. The goal for the Pointmass environment is to navigate a grid world of varying difficulties: *easy*, *medium*, and *hard* to reach the ‘goal’ location. Sample environments for each difficulty have been visualized in Figure 1. The observation consists of (y, x) -location of the point mass and the action space is 5 discrete numbers indicating NO MOVE, LEFT, RIGHT, UP, and DOWN. The agent receives 1 reward and terminates when the agent reaches the goal, otherwise receives reward of 0.

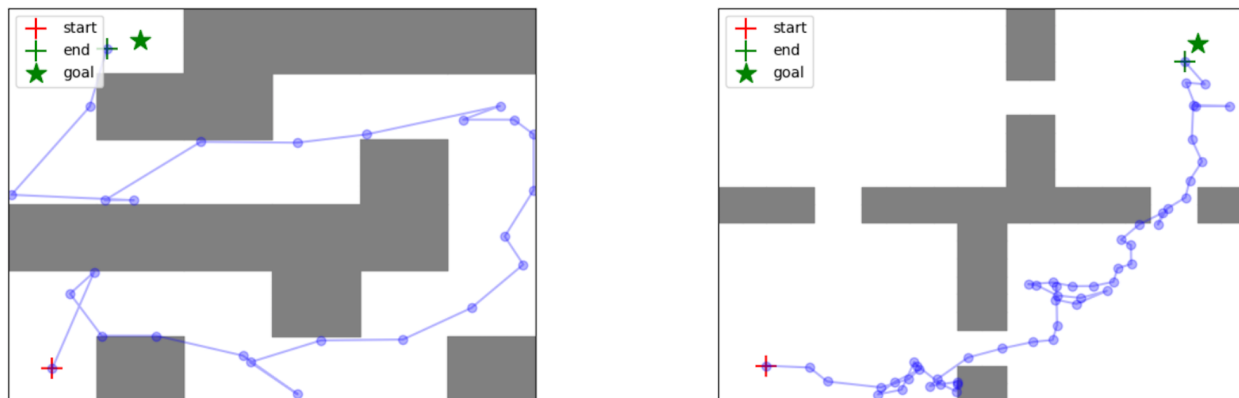


Figure 1: Visualization of the Pointmass environments with Medium (left) and Hard (right) difficulties for navigation and reaching the goal.

Offline datasets: For offline RL, we have provided you with a set of data collection strategies which are used to collect the offline dataset. These trajectories then serve as the training dataset for your offline RL algorithms. The offline transitions dataset $\mathcal{D} = \{(s_i, a_i, r_i, s_{i+1})_i\}$ consists of tuples of the current state, action, reward, and future state. In this assignment, you will experiment with two datasets generated by (1) random policy (100000 steps) and (2) expert policy (20000 steps).

5 Conservative Q-Learning

5.1 Implement CQL

In this problem, you’ll be implementing the **Conservative Q-Learning** (CQL) algorithm (a DQN version for the discrete environment). The goal of CQL is to prevent overestimation of the policy value. The overall CQL objective is given by the standard TD error objective augmented with the CQL regularizer weighted by α :

$$L_Q(\theta) = \frac{1}{N} \sum_{i=1}^N \left[\left(Q(s_i, a_i) - \left(r(s_i, a_i) + \gamma \max_{a'} [Q(s'_i, a')] \right) \right)^2 \right] + \alpha \frac{1}{N} \sum_{i=1}^N \left[\log \left(\sum_{\mathbf{a}} \exp(Q(s_i, \mathbf{a})) \right) - Q(s_i, a_i) \right]. \quad (1)$$

Fill in the TODOs in the following file:

- `cas4160/critics/cql_critic.py`

Once you’ve filled in all of the TODO commands, you should be able to run CQL on the **expert** dataset with the following command:

```
python cas4160/scripts/run_cql.py --env_name PointmassMedium-v0 \
--exp_name cql_alpha_{$ALPHA}_expert \
--dataset_name expert --cql_alpha={$ALPHA}
```

Run CQL with $\alpha = \{0.0, 0.1\}$ in expert dataset. Please report the Eval AverageReturn, Eval StdReturn of the last step in your tensorboard log file in the table. Your CQL implementation should achieve about at least 0.9 in expert dataset with $\alpha = 0.1$ in Medium difficulty. The code will run for 50000 iterations and takes around 30 minutes. Also, attach the eval_last_traj.png image, which visualizes your agent's last trajectory, and the *overestimation* value curve, $\frac{1}{N} \sum_{i=1}^N [\log(\sum_{\mathbf{a}} \exp(Q(s_i, \mathbf{a}))) - Q(s_i, \mathbf{a}_i)]$.

What does $\alpha = 0.0$ signify in terms of the algorithm (hint: think about an algorithm on top of which we developed CQL)? Explain the difference in the performance gap and the overestimation value with respect to α .

Answer: Include your answer here.

$\alpha = 0.0$ means multiplying a weight of 0 to the CQL regularizer term ($\alpha \times [\log \sum_{\mathbf{a}} \exp(Q(s_i, \mathbf{a})) - Q(s_i, \mathbf{a}_i)]$). Therefore, the regularizer completely disappears from the CQL objective, making it exactly the same algorithm as the existing vanilla DQN (or soft Q-learning). CQL is an algorithm developed by adding a conservative regularizer on top of vanilla DQN.

Performance differences and differences in overestimation values according to α :

- In the Expert dataset, when $\alpha = 0.0$, the overestimation value increases significantly, and the Q-value for unseen actions is evaluated excessively high, leading to policy instability. On the other hand, when the regularizer is applied with $\alpha = 0.1$, overestimation is effectively suppressed, resulting in a significant improvement in Eval Average Return (achieving approximately 0.9 or higher).
- In the Random dataset, since the data itself consists mostly of suboptimal or random actions, the overestimation problem for unseen actions is relatively less severe. Therefore, the performance difference between $\alpha = 0.0$ and $\alpha = 0.1$ is not as large as in the expert dataset. Even $\alpha = 0.0$ alone already shows considerable performance, but using $\alpha = 0.1$ can still control overestimation more stably.

Overall, it can be confirmed that the higher the data quality (expert), the more significant the effect of the CQL regularizer ($\alpha > 0.0$) becomes.

Table 1: Average performance of CQL on the expert dataset with different values of α (PointmassMedium-v0 environment).

	Mean	Standard Deviation
$\alpha = 0.0$	0	0
$\alpha = 0.1$	1	0

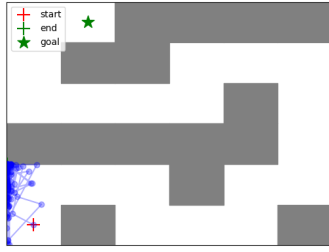


Figure 2: Last evaluation trajectory of the learned policy on the expert dataset with $\alpha = 0.0$ (PointmassMedium-v0).

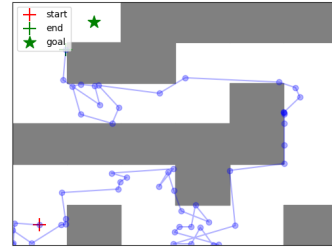


Figure 3: Last evaluation trajectory of the learned policy on the expert dataset with $\alpha = 0.1$ (PointmassMedium-v0).

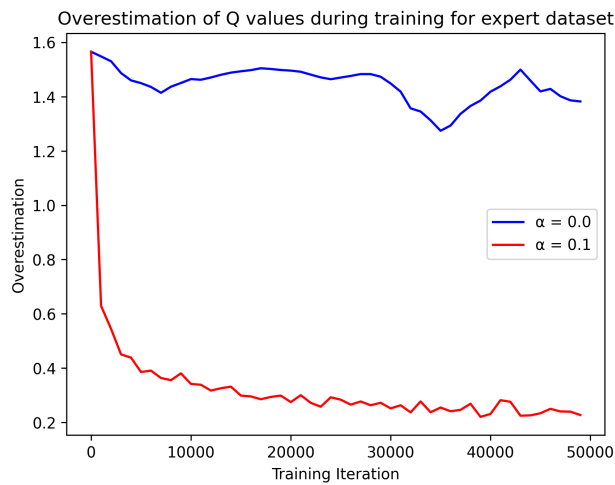


Figure 4: Overestimation of Q-values (CQL regularizer / CQL Loss) during training on the expert dataset with $\alpha = 0.0$ and $\alpha = 0.1$.

5.2 Comparison between Expert dataset and Random dataset (5 points)

Compare the learned policies for the expert dataset and random dataset on the PointmassMedium-v0 environment with $\alpha = \{0.0, 0.1\}$ by reporting the eval average return and standard deviation. Execute the below command to execute CQL with **random** dataset:

```
python cas4160/scripts/run_cql.py --env_name PointmassMedium-v0 \
  --exp_name cql_alpha_${ALPHA}_random \
  --dataset_name random --cql_alpha=${ALPHA}
```

Attach the `eval_last_traj.png` image denoting your agent's last trajectory for each run.

You may have noticed that the performance gap between $\alpha = 0.0$ and $\alpha = 0.1$ is not as large in the random dataset as it was in the previous experiment with the expert dataset. Explain why $\alpha = 0.0$ performs well on the random dataset but fails on the expert dataset. Hint: CQL aims to prevent overestimation of 'unseen' actions.

Answer: Include your answer here.

In the Random dataset, the performance difference between $\alpha = 0.0$ and $\alpha = 0.1$ was not as large as that observed in the expert dataset.

$\alpha = 0.0$ is equivalent to vanilla DQN with the CQL regularizer completely removed. Because the Expert dataset consists of high-quality (near-optimal) actions, the Q-values for unseen actions become excessively overestimated. As a result, with $\alpha = 0.0$, the policy becomes unstable and the Eval Average Return decreases. On the other hand, applying the CQL regularizer ($\alpha > 0$) actively suppresses the Q-values of unseen actions, greatly improving performance.

However, the Random dataset is primarily composed of suboptimal or random actions, so the overestimation problem for unseen actions is relatively less severe compared to the expert dataset. Therefore, even $\alpha = 0.0$ alone can already learn a fairly stable policy, and applying $\alpha = 0.1$ does not yield a significant additional performance improvement.

This demonstrates that CQL’s core idea of “preventing overestimation for unseen actions” depends heavily on the quality of the data. The more random the data, the lower the necessity of the regularizer becomes; conversely, the closer the data is to expert level, the more significant the effect of the regularizer ($\alpha > 0$) becomes.

Table 2: Average performance comparison of CQL between expert and random datasets with different values of α (PointmassMedium-v0 environment).

	Mean	Standard Deviation
Random; $\alpha = 0.0$	1	0
Random; $\alpha = 0.1$	1	0
Expert; $\alpha = 0.0$	0	0
Expert; $\alpha = 0.1$	1	0

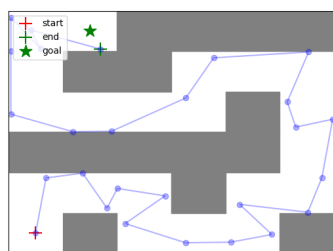


Figure 5: Last evaluation trajectory of the learned policy on the random dataset with $\alpha = 0.0$ (PointmassMedium-v0).

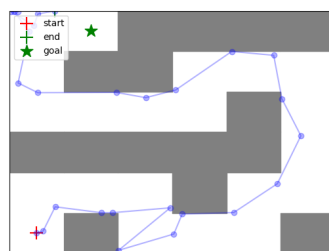


Figure 6: Last evaluation trajectory of the learned policy on the random dataset with $\alpha = 0.1$ (PointmassMedium-v0).

6 Submission

Please include the code and the report in a single zip file, hw5_[YourStudentID].zip. Do not include videos and logs as the file size should be less than 20MB. The structure of the submission file should be:

```
hw5_[YourStudentID].zip
├── hw5_[YourStudentID].pdf
├── cas4160
│   ├── critics
│   │   ├── cql.critic.py
│   │   └── ...
│   └── ...
└── ...
```